

Beta System — Document Generation Analysis

- Document Generation: Legacy System vs. LawDocs
 - Purpose
 - 1. Document Generation Pipeline (Beta)
 - 2. PDF Export Capability (Beta)
 - 3. Document Delivery Mechanism (Beta)
 - 4. Side-by-Side Comparison: Document Generation and Delivery
 - 5. Full System Comparison

Document Generation: Legacy System vs. LawDocs

Purpose

This document analyzes how the firm’s previous system (“beta”) generated documents, with particular attention to PDF export, and compares that architecture to the current LawDocs application. It concludes with a candid assessment of the strengths and weaknesses of each system.

Summary finding: Beta never produced PDF output. It generated Word (`.docx`) documents exclusively. LawDocs introduces PDF export as a new capability, not a restoration of prior functionality.

1. Document Generation Pipeline (Beta)

Beta's generation flow proceeds as follows:

1. A staff member completes a form on the beta site and submits it.
2. `handlesubmission.php` passes the request to `controllers/handleSubmissionController.php`.
3. The controller calls `SubmissionHandler::submitForm()` in `services/FormSubmission.php`, the single entry point for every document generation in beta.
4. `submitForm()` looks up the PHP script registered for that form (a database column, `OutputGenerationAddress`) and includes it directly — for example, `OutputGenerationAddress/TestUser/TU_CostAgreement.php` for the Costs Agreement form.
5. That per-form script builds the document by hand, calling the PHPWord library's API directly (`addSection()`, `addTextRun()`, `addTable()`, `addListItem()`, and similar methods). Each document type is one large procedural script; there is no shared generator abstraction.
6. Control returns to `submitForm()`, which saves the finished document twice: once to a permanent folder on the server (`OutputFileLocation/<Client>/...docx`, retained indefinitely), and once again, immediately, streamed to the browser as a forced download.

The entire process is synchronous, within a single web request. There is no queue, background job, or scheduled processing.

2. PDF Export Capability (Beta)

Beta never produced PDF output. Every save in the live code path is hardcoded to the `'Word2007'` format; `'PDF'` is never passed. A review of beta's entire application code for PDF-related functionality (`soffice`, `libreoffice`, `dompdf`, `mpdf`, `tcpdf`, `wkhtmltopdf`, or any external shell-out) found none. Of the roughly forty sample documents present in beta's output folder, none are PDFs.

The underlying PHPWord library beta bundles does include a PDF-writer subsystem, capable of delegating to one of three third-party rendering libraries (DomPDF, mPDF, or TCPDF). However:

- Those three libraries are listed only as PHPWord's own test-suite dependencies. They were never installed in beta's production `vendor/` folder.
- No application code in beta ever invokes the PDF writer.

Had a developer attempted to add PDF export by changing the format argument from `'Word2007'` to `'PDF'`, the change would have failed immediately with a missing-class error, since the required rendering library is not present.

Conclusion: PDF export in beta is dormant, unused capability within a bundled library. It was never a working feature, was never offered to users, and its absence in the current transition does not represent a regression.

3. Document Delivery Mechanism (Beta)

Beta's copy of PHPWord is patched — not the stock library — to add a third parameter to its `save()` method: a `$download` flag. When enabled, the method sends the appropriate browser headers (`Content-Disposition: attachment` , and related headers) and streams the file directly into the HTTP response. This patched method is what allows both the permanent-copy save and the forced-download response to originate from the same generated document within `submitForm()` .

Beta also contains code to email the generated document to the client — a complete HTML email template with attachment, built using PHPMailer — but the line that triggers it is commented out. The confirmation message shown to staff states that the file “has been emailed,” even though no email is sent. This is a pre-existing defect in beta, unrelated to PDF handling.

4. Side-by-Side Comparison: Document Generation and Delivery

	Beta (Legacy)	Current LawDocs
Word processing engine	PHPWord 0.18.3, a locally patched copy	PHPWord 1.4.0, standard package
Document construction	One procedural PHP script per document type, calling the PHPWord API directly	A shared clause-marker system (<code>[[CLAUSE:...]]</code> , <code>[[IF:...]]</code> , <code>[[REPEAT:...]]</code>) editable by re-uploading a <code>.docx</code> — no PHP script required per document type
PDF export	Never implemented; no external dependency, as no PDF was ever produced	Implemented: converts the generated <code>.docx</code> to PDF via LibreOffice (<code>soffice --convert-to pdf</code>)
PDF server requirement	None	LibreOffice must be installed on the server
Delivery	Saved to a server folder and forced-downloaded in the same request	Saved to storage; downloaded (forced) or previewed (inline, in-browser) on demand
Email delivery	Present in code but disabled	Not covered by this comparison

Because beta never solved document-to-PDF conversion, there is no prior working mechanism to fall back on. The LibreOffice dependency introduced by LawDocs is a new capability, not a regression of one that previously worked.

5. Full System Comparison

Beta is an unframed PHP application (1,845 files, direct PDO database access, no ORM, vendored libraries copied in by hand). LawDocs is built on Laravel and Filament, with a structured admin panel, role-based permissions, and an ORM. The comparison below covers the system as a whole, not only document generation.

5.1 Document Type Configuration

	Beta	Current LawDocs
Adding a new document type	A developer writes a new PHP script by hand, calling the PHPWord API directly (e.g. <code>TU_CostAgreement.php</code> , over 1,300 lines)	An administrator uploads a <code>.docx</code> tagged with <code>[[CLAUSE:...]]</code> markers. This covers most document types, including conditional and fallback wording (for example, falling back to an alternative appointee when none is named, with pronouns resolved automatically) that previously required a hand-written PHP method. A no-code “Template” generator supports building an entirely new document type through configuration alone
Changing existing wording	A developer edits the PHP script and redeploys	An administrator re-uploads a revised <code>.docx</code> ; changes take effect immediately, with no deployment
Reordering or hiding sections	Requires editing the PHP script’s logic	Configurable via a “Structure” tab: sections can be reordered, renamed, or conditionally hidden
Fonts and heading style	Hardcoded in the PHP script	Configurable per precedent (font, size, heading weight) via a “Formatting” tab

This is the most significant structural difference between the two systems, and the primary motivation for the LawDocs rebuild: the requirement for developer involvement in every wording change is addressed in the common case under LawDocs, and is not addressed at all in beta.

5.2 Questionnaire and Input Field Management

	Beta	Current LawDocs
Adding a single field	Insert a row into a <code>forminput</code> - style table via an admin form	Add a row via a repeater on the precedent’s “Questionnaire Fields” tab

	Beta	Current LawDocs
Adding fields in bulk	Bulk CSV import — a spreadsheet of <code>FormID</code> , <code>InputName</code> , <code>InputLabel</code> , <code>InputType</code> , ... creates every field in one pass	Bulk CSV import is also supported, validating each row and reporting any that fail rather than rejecting the file
Repeatable fields (e.g. multiple beneficiaries)	Fixed-count, pre-declared slots (e.g. <code>WorkDescription01</code> – <code>WorkDescription10</code>), or a <code>TableNumber</code> / <code>TableRow</code> / <code>TableColumn</code> grid convention, capped at whatever was pre-built	A dynamic repeatable “Party Group” with configurable minimum and maximum, with no fixed cap on field count

Bulk CSV import for field definitions was the one area where beta was ahead of LawDocs when this comparison was first conducted. That gap has since been closed.

5.3 Document Generation Workflow

	Beta	Current LawDocs
Generating a single document	Complete the form, submit, and receive an immediate forced download	Complete the request wizard, preview as needed with no side effects, then submit
Generating multiple documents	A CSV-batch mode exists but has no per-row validation or error reporting; a single invalid row can interrupt the entire batch, and the multi-document download response does not reliably deliver more than one file	A dedicated batch-generation flow validates every row before processing, reports what will succeed or be skipped, and creates each document as its own trackable request (up to 200 per file)
Review before finalization	Not available; the file downloads immediately on submission	A preview is available both before and after generation, including before a reviewer approves the document
Approval workflow	None; a generated file is immediately downloadable and considered final	Precedents can be flagged as requiring review; a completed draft remains pending until approved by an authorized user

5.4 Format Support

As detailed in Sections 1–4, beta supports Word output only. LawDocs additionally supports PDF export, which requires LibreOffice on the server.

5.5 Delivery, Notifications, and Audit Trail

	Beta	Current LawDocs
Delivery	Saved to a server folder and force-downloaded within the same request	Saved to storage; downloaded or previewed independently of generation. PDF output is cached after first conversion rather than regenerated on each request
Email	Present in code (PHPMailer, full HTML template) but never triggered; the confirmation message is inaccurate	In-app notifications for document readiness and review status; a separate email alert on generation failure
Audit trail	A basic log table records who generated a document, when, and where it was saved; no other activity is recorded	Every significant state change (creation, edits, approvals, signature status) is recorded in an activity log attributable to a specific user

5.6 Access Control

Beta's access model was not reviewed in exhaustive depth, but nothing in its codebase resembles role-based access control; access appears to be governed by authentication status alone. LawDocs uses a role-based permission system with named roles (administrator, operator, panel user) and granular per-action permissions, enforced through a consistent policy layer.

5.7 Summary of Strengths and Weaknesses

Beta — strengths

- Simple and direct: no framework overhead for a developer working directly in PHP.
- Bulk CSV import for defining input fields was a genuinely effective approach, and has been carried forward into LawDocs.
- Minimal operational footprint: no queue system, permission tables, or ORM overhead.

Beta — weaknesses

- Every new document type, and every wording change, requires a developer and a deployment.
- No PDF export, no pre-submission preview, no approval workflow, and no audit trail beyond a record of who generated a given file.
- Batch document generation is unreliable: no per-row validation, a single invalid row can interrupt the whole batch, and multi-document delivery does not function correctly.
- A defect in the confirmation message tells users a document was emailed when the email functionality is disabled.
- Direct, hand-written database queries and no access-control layer, increasing the risk of error as the system grows.

Current LawDocs — strengths

- Wording changes, structural changes, and the creation of entirely new document types do not require developer involvement.
- A functioning review workflow: a reviewer can read a draft before approving it, rather than relying on trust alone.
- PDF export, unlimited dynamic repeatable fields, bulk CSV import for both field definitions and batch document generation, comprehensive activity logging, and role-based permissions.
- Batch generation is validated before execution and reports outcomes before any document is created.

Current LawDocs — weaknesses

- The system has more operating parts overall — a full framework, permission tables, activity-log tables, and a queue system — requiring more operational understanding than beta's simpler footprint, though this complexity underlies the capabilities described above.